

LAPORAN TUGAS BESAR

Implementasi Neural Network from Scratch: Klasifikasi Dry Bean Dataset

Mata Kuliah Kecerdasan Buatan

Dosen Pengampu :

Dr. Ing Ahmad Zainul Ihsan, S.T., M.Sc.



Disusun Oleh :

Ahmad Aqil Almutashim (2403029028)

Suhaila Ade Syafiq (2403020046)

Hanif Hidayatullah (2403020041)

Noufal Henri Putra Hermawan (2403020040)

PROGRAM STUDI

REKAYASA PERANGKAT LUNAK

UNIVERSITAS DARUNNAJAH

2025-2026

DAFTAR ISI

BAB I

DESKRIPSI PERSOALAN

Tugas besar ini dirancang untuk mata kuliah Kecerdasan Buatan dan bertujuan untuk membangun Artificial Neural Network (ANN) dari nol menggunakan Python dan NumPy tanpa menggunakan framework seperti TensorFlow atau PyTorch untuk bagian inti jaringannya. Dataset yang dipilih adalah Dry Bean Dataset dari UCI Machine Learning Repository, sebuah dataset klasifikasi multi-kelas yang datanya berasal dari sistem visi komputer untuk membedakan tujuh varietas biji kacang kering.

Persoalan yang dihadapi cukup menantang: bagaimana membangun seluruh mekanisme neural network — mulai dari inisialisasi bobot, forward propagation, fungsi loss, hingga backpropagation — hanya dengan operasi matriks NumPy, lalu membuktikan bahwa hasilnya bisa bersaing dengan implementasi Keras yang jauh lebih mudah digunakan. Di sisi lain, bekerja with dataset biji kacang yang punya 7 kelas dan 16 fitur morfologi juga menjadi tantangan tersendiri dalam hal preprocessing dan evaluasi

A. Latar Belakang

Artificial Neural Network (ANN) merupakan salah satu Teknik pembelajaran mesin yang terinspirasi dari cara kerja neuron biologis pada otak manusia. ANN terdiri dari lapisan-lapisan (layers) neuron buatan yang saling terhubung, dimana setiap koneksi memiliki bobot (weight) yang dipelajari melalui proses Latihan.

Dalam konteks klasifikasi Dry Bean Dataset , ANN digunakan untuk memetakan fitur morfologi biji kacang kedalam salah satu dari tujuh kelas varietas. Pemahaman mendalam tentang cara kerja ANN sangat penting sebelum menggunakan framework high-level, karena hal tersebut memungkinkan pengembang untuk melakukan debugging yang efektif, memodifikasi arsitektur sesuai kebutuhan, dan memahami batasan serta kelebihan dari setiap komponen.

B. Dry Bean Dataset

Dataset yang digunakan adalah Dry Bean Dataset yang dipublikasikan oleh Koklu dan Ozkan (2020) dari Selcuk University, Turki. Dataset ini berasal dari hasil ekstraksi citra digital terhadap 13.611 butir biji kacang kering menggunakan metode computer vision dan image

processing. Setiap baris data merepresentasikan satu biji kacang dengan 16 fitur numerik yang menggambarkan karakteristik geometris dan bentuk objek, seperti luas area, keliling, panjang sumbu utama dan minor, rasio aspek, eccentricity, hingga berbagai shape factor.

No	Nama Fitur	Deskripsi
1	Area	Luas area biji kacang
2	Perimeter	Keliling objek kacang
3	MajorAxisLength	Panjang sumbu utama
4	MinorAxisLength	Panjang sumbu minor
5	Aspectratio	Rasio panjang dan lebar
6	Eccentricity	Tingkat keovalan objek
7	Convex area	Luas convex hull
8	Equivdiameter	Diameter ekuivalen
9	Extent	Rasio area terhadap bounding box
10	Solidity	Kepadatan bentuk objek
11	Roundness	Tingkat kebulatan
12	Compactness	Tingkat kekompakan
13	Shapefaktor1-4	Factor bentuk pertama hingga ke empat

Table 1. Deskripsi fitur pada Dry Bean Dataset

Dataset ini terdiri atas 7 kelas target yang direpresentasikan dalam bentuk one-hot encoding, dengan rincian sebagai berikut:

No	Jenis Kacang	Jumlah Sample
0	Seker	2.027
1	Barbunya	1.322
2	Bombay	522
3	Cali	1.630
4	Dermason	3.546
5	Horoz	1.928
6	Sira	2.636

Table 2. Distribusi kelas pada Dry Bean Dataset

C. Tujuan

Ada beberapa hal yang ingin dicapai dalam tugas ini

- Pertama, mengimplementasikan ANN from scratch yang fungsional menggunakan NumPy, lengkap dengan berbagai fungsi aktivasi, metode inisialisasi bobot, dan mekanisme regularisasi.
- Kedua, melakukan serangkaian eksperimen sistematis untuk memahami bagaimana setiap hyperparameter — arsitektur, learning rate, batch size, hingga jumlah epoch — mempengaruhi performa model.
- Ketiga, membandingkan hasilnya dengan implementasi Keras dan MLPClassifier dari scikit-learn untuk melihat seberapa jauh gap-nya

BAB II

PEMBAHASAN

A. Penjelasan Implementasi

Implementasi ANN from scratch dibangun menggunakan paradigma object-oriented programming (OOP) dengan memisahkan setiap komponen fungsional ke dalam kelas tersendiri. Pemisahan ini bertujuan agar kode lebih modular, mudah diuji, dan memudahkan proses eksperimen karena setiap komponen (inisialisasi bobot, fungsi aktivasi, fungsi loss) dapat ditukar tanpa mengubah struktur utama jaringan.

Secara garis besar, alur implementasi terdiri dari lima tahap utama: (1) preprocessing data, (2) inisialisasi bobot dan bias, (3) forward propagation, (4) perhitungan loss, dan (5) backpropagation beserta update bobot menggunakan Mini-batch Gradient Descent. Kelima tahap ini diulang secara iteratif selama sejumlah epoch yang ditentukan.

B. Deskripsi Kelas, Atribut, dan Method

1. Kelas Weightinitializer

Kelas ini berisi kumpulan static method untuk menginisialisasi bobot jaringan dengan lima metode berbeda. Seluruh method bersifat static karena tidak memerlukan instance/state tersendiri, hanya menerima parameter shape (ukuran matriks bobot) dan mengembalikan array NumPy.

Method	Parameter	Penjelasan
<code>zero(shape)</code>	shape	Mengembalikan matriks bobot bernilai nol semua (<code>np.zeros</code>).
<code>random_uniform(shape, seed)</code>	shape, seed	Bobot diambil dari distribusi uniform pada rentang $[-1, 1]$.
<code>random_normal(shape, seed)</code>	shape, seed	Bobot diambil dari distribusi normal standar ($\text{mean}=0$, $\text{std}=1$).
<code>xavier(shape, seed)</code>	shape, seed	Bobot uniform dengan limit = $\sqrt{6/(n_{\text{in}}+n_{\text{out}})}$, cocok untuk Sigmoid/Tanh.
<code>he(shape, seed)</code>	shape, seed	Bobot normal dikalikan $\text{std} = \sqrt{2/n_{\text{in}}}$, cocok untuk ReLU.

Tabel 1.

pada kelas `WeightInitializer`

Method

2. Kelas Activation

Kelas ini menyediakan empat fungsi aktivasi beserta turunannya, yang digunakan baik pada saat forward propagation (menghitung output neuron) maupun backward propagation (menghitung gradien lokal).

Method	Rumus	Kegunaan
<code>sigmoid(z)</code>	$1 / (1 + e^{-z})$	Aktivasi hidden layer, output ke rentang (0,1).
<code>sigmoid_derivative(z)</code>	$s(z) * (1 - s(z))$	Gradien untuk backpropagation.
<code>relu(z)</code>	$\max(0, z)$	Aktivasi hidden layer, cepat konvergen.
<code>relu_derivative(z)</code>	1 jika $z > 0$, 0 jika $z \leq 0$	Gradien untuk backpropagation.
<code>tanh(z)</code>	$\tanh(z)$	Aktivasi hidden layer, output ke rentang (-1,1).
<code>tanh_derivative(z)</code>	$1 - \tanh(z)^2$	Gradien untuk backpropagation.
<code>softmax(z)</code>	$e^z / \sum(e^z)$	Aktivasi output layer untuk klasifikasi multiclass.

Tabel 2. Method pada kelas Activation

Pada implementasi sigmoid, digunakan `np.clip` untuk membatasi nilai input ke rentang [-500, 500] guna mencegah overflow numerik (exploding value) pada fungsi eksponensial. Pada softmax, nilai dikurangi dengan nilai maksimum pada axis yang sesuai ($z - \max(z)$) sebelum diekspensialkan, sebagai teknik numerical stability standar untuk mencegah overflow.

3. Kelas LossFunction

Kelas ini menyediakan tiga fungsi loss yang berbeda sesuai kebutuhan jenis klasifikasi, beserta turunannya masing-masing yang digunakan sebagai titik awal proses backpropagation.

Method	Kegunaan
<code>mse(y_true, y_pred)</code>	Mean Squared Error, umum digunakan pada regresi.
<code>binary_crossentropy(y_true, y_pred)</code>	Loss untuk klasifikasi biner (2 kelas).
<code>categorical_crossentropy(y_true, y_pred)</code>	Loss untuk klasifikasi multiclass (digunakan pada tugas ini).

Tabel 3. Method pada kelas LossFunction

Pada seluruh fungsi loss berbasis cross entropy, nilai y_{pred} dibatasi (clipped) ke rentang $[1e-15, 1-1e-15]$ sebelum dimasukkan ke dalam fungsi logaritma, untuk menghindari hasil $\log(0)$ yang tidak terdefinisi.

4. Kelas ANN

Kelas ANN merupakan kelas utama yang menggabungkan seluruh komponen di atas menjadi satu kesatuan jaringan saraf tiruan yang dapat dilatih. Kelas ini menyimpan state berupa daftar bobot (weights) dan bias (biases) untuk setiap layer, serta menyimpan hasil komputasi sementara (a dan z) yang dibutuhkan saat backpropagation.

Atribut/Method	Tipe	Penjelasan
<code>layer_sizes</code>	Atribut	List jumlah neuron tiap layer, contoh [16, 64, 32, 7].
<code>weights, biases</code>	Atribut	List matriks bobot dan vektor bias tiap layer.
<code>a, z</code>	Atribut	Cache aktivasi (a) dan nilai pre-aktivasi (z) tiap layer hasil forward pass, dipakai ulang saat backward.
<code>_initialize_weights()</code>	Method	Menginisialisasi weights dan biases sesuai <code>init_method</code> yang dipilih.
<code>forward(X)</code>	Method	Melakukan forward propagation, mengembalikan output prediksi.
<code>backward(X, y_true, lr)</code>	Method	Melakukan backpropagation dan memperbarui bobot.
<code>predict(X)</code>	Method	Alias dari <code>forward</code> , digunakan saat inferensi.
<code>accuracy(X, y_true)</code>	Method	Menghitung akurasi klasifikasi pada data yang diberikan.

Tabel 4. Atribut dan method pada kelas ANN

C. Penjelasan Forward Propagation

Forward propagation adalah proses menghitung output jaringan dari input yang diberikan, dengan mengalirkan data dari input layer menuju output layer melalui setiap hidden layer secara berurutan. Pada setiap layer ke- i , dilakukan dua langkah komputasi berikut:

1. Menghitung nilai pre-aktivasi: $z[i] = a[i-1] \cdot W[i] + b[i]$, yaitu hasil kali matriks antara aktivasi layer sebelumnya dengan matriks bobot, ditambah bias.
2. Menerapkan fungsi aktivasi: $a[i] = f(z[i])$, di mana f adalah ReLU, Sigmoid, atau Tanh untuk hidden layer, dan Softmax untuk output layer.

Nilai $a[0]$ sendiri merupakan data input X . Proses ini diulang untuk seluruh layer hingga mencapai output layer, yang menggunakan fungsi aktivasi Softmax agar menghasilkan distribusi probabilitas terhadap 7 kelas kacang yang berjumlah total 1. Seluruh nilai $z[i]$ dan $a[i]$ pada setiap layer disimpan dalam cache ($self.z$ dan $self.a$) karena nilai-nilai tersebut akan dibutuhkan kembali pada proses backpropagation.

D. Penjelasan Backward Propagation dan Weight Update

Backpropagation adalah proses menghitung gradien loss terhadap setiap bobot dan bias di jaringan, menggunakan prinsip chain rule dari kalkulus, dengan arah komputasi dari output layer mundur ke input layer. Proses ini terdiri dari langkah-langkah berikut:

1. Menghitung delta pada output layer, yaitu turunan loss terhadap output prediksi. Untuk kombinasi Softmax dan Categorical Cross Entropy, turunan ini dapat disederhanakan menjadi $\text{delta} = (y_{\text{pred}} - y_{\text{true}}) / m$, dengan m adalah jumlah sampel pada batch.
2. Untuk setiap layer dari belakang ke depan, dihitung gradien bobot $dW = a[i].T \cdot \text{delta}$ dan gradien bias $db = \text{sum}(\text{delta}, \text{axis}=0)$.
3. Jika layer bukan merupakan input layer, delta diteruskan ke layer sebelumnya dengan formula: $\text{delta_baru} = (\text{delta} \cdot W[i].T) * f'(z[i-1])$, di mana f' adalah turunan fungsi aktivasi pada layer tersebut. Perkalian dengan turunan aktivasi inilah yang menjadi inti dari chain rule pada backpropagation.
4. Setelah seluruh gradien dihitung, bobot dan bias diperbarui menggunakan aturan Gradient Descent: $W[i] = W[i] - \text{learning_rate} * dW$, dan $b[i] = b[i] - \text{learning_rate} * db$.

Proses update bobot ini dilakukan secara bertahap per mini-batch (Mini-batch Gradient Descent), bukan per satu sampel (Stochastic Gradient Descent) ataupun seluruh dataset sekaligus (Batch Gradient Descent). Pada setiap epoch, data latih diacak ulang (shuffle) kemudian dibagi menjadi beberapa mini-batch berukuran tetap (misalnya 32 sampel), dan untuk setiap mini-batch dilakukan satu kali forward propagation dan satu kali backward propagation. Pendekatan ini dipilih karena memberikan keseimbangan antara stabilitas gradien (dibandingkan SGD murni) dan efisiensi komputasi (dibandingkan Batch GD murni).

E. Fungsi Training (Mini-Batch Gradient Descent)

Fungsi `train()` bertugas mengorkestrasi seluruh proses pelatihan model selama sejumlah epoch. Pada setiap epoch, fungsi ini melakukan pengacakan urutan data, membagi data ke dalam mini-batch, memanggil forward dan backward untuk setiap mini-batch, kemudian menghitung dan mencatat metrik loss serta akurasi pada data training maupun testing untuk keperluan visualisasi dan analisis. Seluruh riwayat metrik disimpan dalam dictionary history yang kemudian digunakan untuk membuat grafik loss vs epoch dan accuracy vs epoch pada setiap eksperimen.

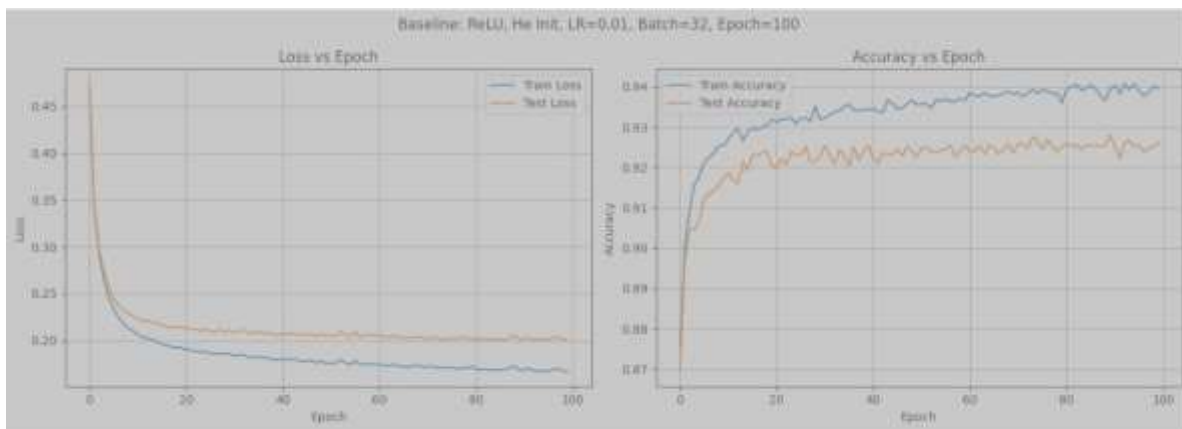
BAB III

HASIL PENGUJIAN

Seluruh eksperimen pada bagian ini menggunakan pengaturan dasar (baseline) berupa arsitektur 2 hidden layer (64 dan 32 neuron), fungsi aktivasi ReLU pada hidden layer dan Softmax pada output layer, inisialisasi bobot He, fungsi loss Categorical Cross Entropy, learning rate 0.01, batch size 32, dan jumlah epoch 100, kecuali pada parameter yang sedang diuji. Pembagian data menggunakan rasio 80% data training (10.888 sampel) dan 20% data testing (2.723 sampel), dengan stratified split untuk menjaga proporsi tiap kelas. Seluruh eksperimen menggunakan random seed 42 agar hasil bersifat reproducible.

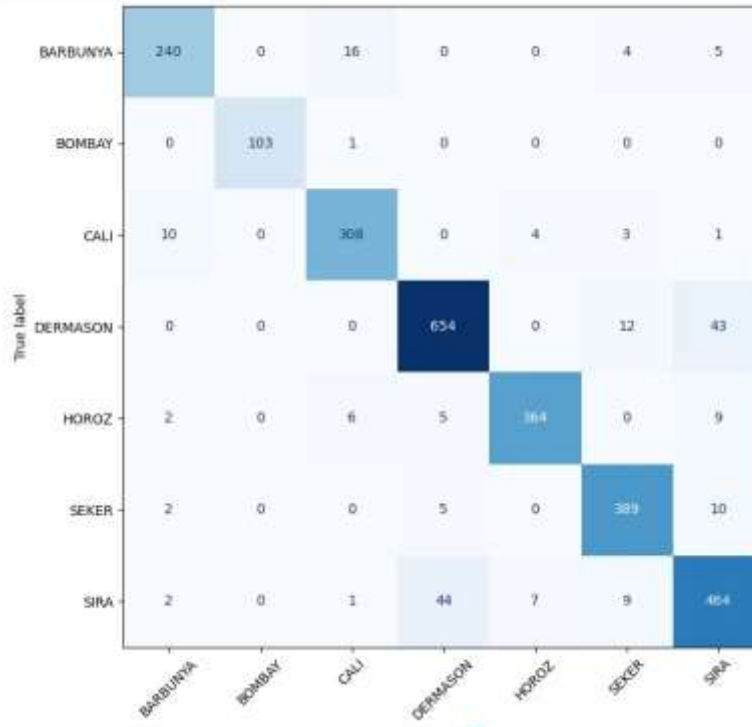
Model Baseline

Sebelum melakukan eksperimen perbandingan, terlebih dahulu dilatih model baseline dengan konfigurasi standar di atas, sebagai titik acuan (reference point) untuk seluruh eksperimen berikutnya.



Gambar 1.1 Grafik loss dan akurasi model baseline (ReLU, He, LR=0.01, Batch=32, Epoch=100)

Model baseline mencapai akurasi training sebesar 93,95% dan akurasi testing sebesar 92,62% setelah 100 epoch. Selisih (gap) antara akurasi training dan testing yang relatif kecil (1,33%) mengindikasikan bahwa model tidak mengalami overfitting yang signifikan pada konfigurasi dasar ini.



Gambar 1.2. Confusion matrix model baseline

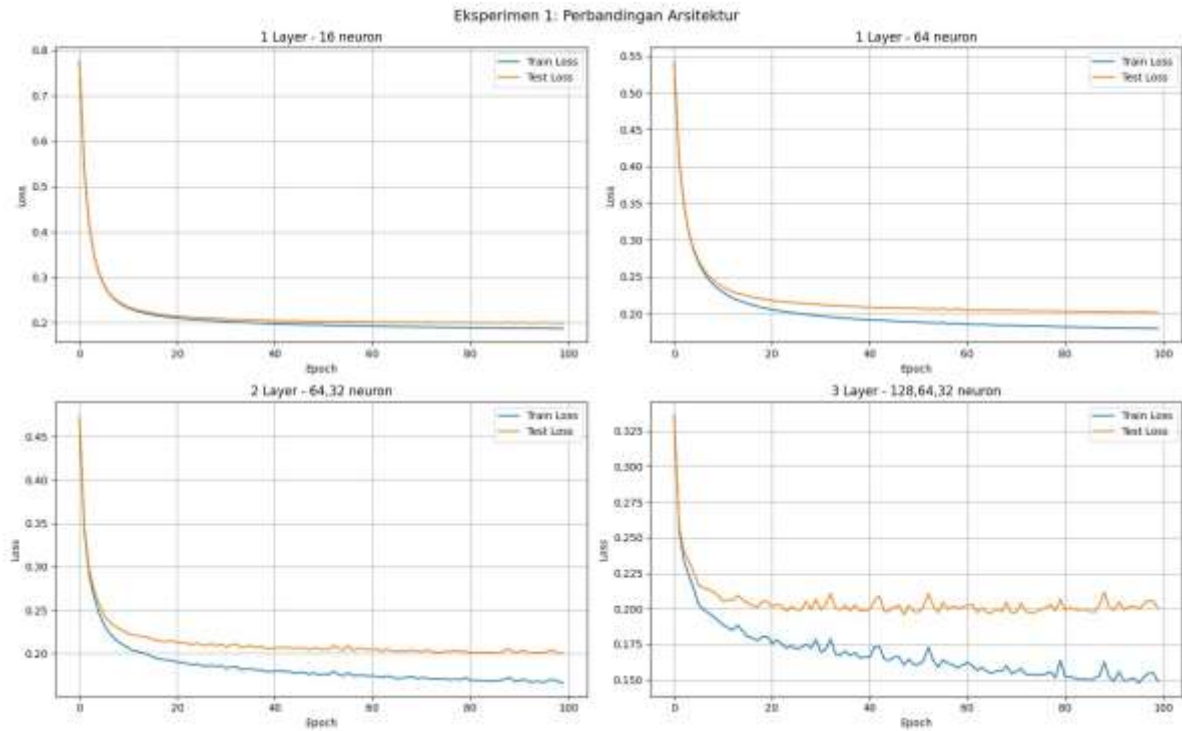
Dari confusion matrix di atas, terlihat bahwa kelas Bombay memiliki tingkat akurasi klasifikasi tertinggi (103 dari 104 sampel diklasifikasikan benar), sementara kesalahan klasifikasi paling banyak terjadi antara kelas Dermason dan Sira, dengan 43 sampel Dermason yang salah diklasifikasikan sebagai Sira. Hal ini mengindikasikan bahwa kedua kelas tersebut memiliki karakteristik bentuk geometris yang relatif serupa.

A. Pengaruh Depth dan Width Jaringan

Eksperimen ini menguji pengaruh jumlah hidden layer (depth) dan jumlah neuron per layer (width) terhadap performa model, dengan parameter lain dijaga konstan mengikuti konfigurasi baseline.

Arsitektur	Train Acc	Test Acc	Train Loss	Test Loss
1 Layer - 16 neuron	0,9327	0,9251	0,1872	0,1993
1 Layer - 64 neuron	0,9344	0,9262	0,1796	0,2014
2 Layer - 64, 32 neuron	0,9395	0,9262	0,1660	0,1998
3 Layer - 128, 64, 32 neuron	0,9441	0,9280	0,1489	0,1999

Tabel 5. Hasil eksperimen variasi arsitektur jaringan



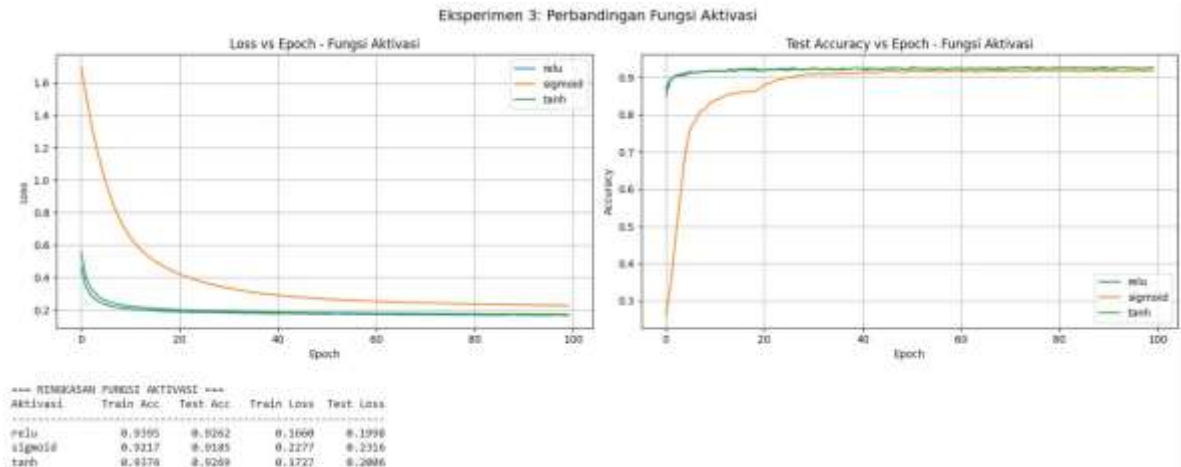
Gambar 1.3. Grafik loss vs epoch untuk setiap konfigurasi arsitektur

Hasil eksperimen menunjukkan bahwa penambahan depth dan width secara umum meningkatkan akurasi, meskipun peningkatannya tidak signifikan secara linear. Arsitektur 3 hidden layer (128, 64, 32 neuron) memberikan akurasi testing tertinggi (92,80%), namun pada grafik loss terlihat train loss menurun jauh lebih dalam (0,1489) dibandingkan test loss yang relatif stagnan (0,1999), serta kurva test loss yang lebih fluktuatif. Pola ini merupakan indikasi awal mulai munculnya overfitting ringan pada arsitektur yang lebih dalam, karena kapasitas model yang lebih besar membuatnya lebih mudah menghafal pola spesifik data training.

Sebaliknya, arsitektur dengan 1 hidden layer dan 16 neuron menghasilkan akurasi testing yang sedikit lebih rendah (92,51%) namun dengan gap train-test yang paling kecil, menunjukkan model yang lebih sederhana ini berada pada kondisi yang mendekati seimbang antara bias dan variance.

B. Pengaruh Fungsi Aktivasi

Eksperimen ini membandingkan tiga fungsi aktivasi pada hidden layer, yaitu ReLU, Sigmoid, dan Tanh, dengan fungsi aktivasi output layer tetap menggunakan Softmax.



Gambar 1.4. Grafik loss dan akurasi untuk setiap fungsi aktivasi

Aktivasi	Train Acc	Test Acc	Train Loss	Test Loss
ReLU	0,9395	0,9262	0,1660	0,1998
Sigmoid	0,9217	0,9185	0,2277	0,2316
Tanh	0,9374	0,9269	0,1727	0,2006

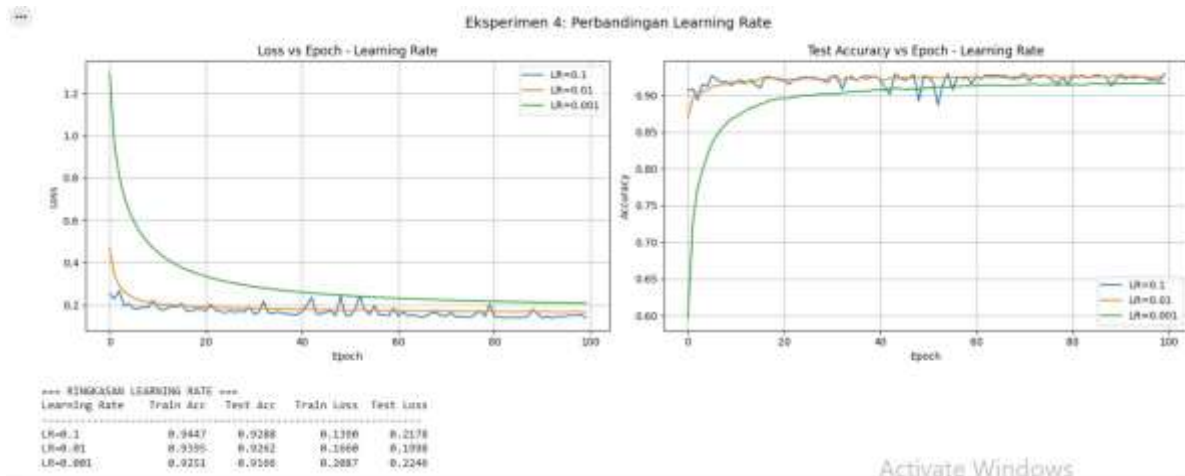
Tabel 6. Hasil eksperimen variasi fungsi aktivasi

Sigmoid menunjukkan kecepatan konvergensi yang jauh lebih lambat dibandingkan ReLU maupun Tanh, terlihat dari kurva loss yang masih tinggi (di atas 1,0) pada epoch-epoch awal dan turun secara perlahan. Hal ini merupakan manifestasi dari vanishing gradient problem, di mana turunan fungsi sigmoid bernilai sangat kecil pada saat input bernilai sangat besar atau sangat kecil, sehingga gradien yang mengalir mundur ke layer-layer awal menjadi semakin mengecil dan memperlambat proses pembelajaran.

ReLU dan Tanh menunjukkan kecepatan konvergensi yang serupa dan jauh lebih cepat dibandingkan Sigmoid, dengan akurasi akhir yang juga lebih tinggi. Tanh memperoleh akurasi testing tertinggi (92,69%), sedikit di atas ReLU (92,62%), namun ReLU memiliki keunggulan komputasi karena perhitungannya lebih sederhana dan tidak melibatkan fungsi eksponensial, sehingga lebih efisien terutama pada jaringan yang lebih besar.

C. Pengaruh Learning Rate

Eksperimen ini membandingkan tiga nilai learning rate, yaitu 0,1, 0,01, dan 0,001, dengan parameter lain mengikuti konfigurasi baseline.



Gambar 1.5. Grafik loss dan akurasi untuk setiap nilai learning rate

Learning Rate	Train Acc	Test Acc	Train Loss	Test Loss
0,1	0,9447	0,9288	0,1390	0,2178
0,01	0,9395	0,9262	0,1660	0,1998
0,001	0,9251	0,9166	0,2087	0,2246

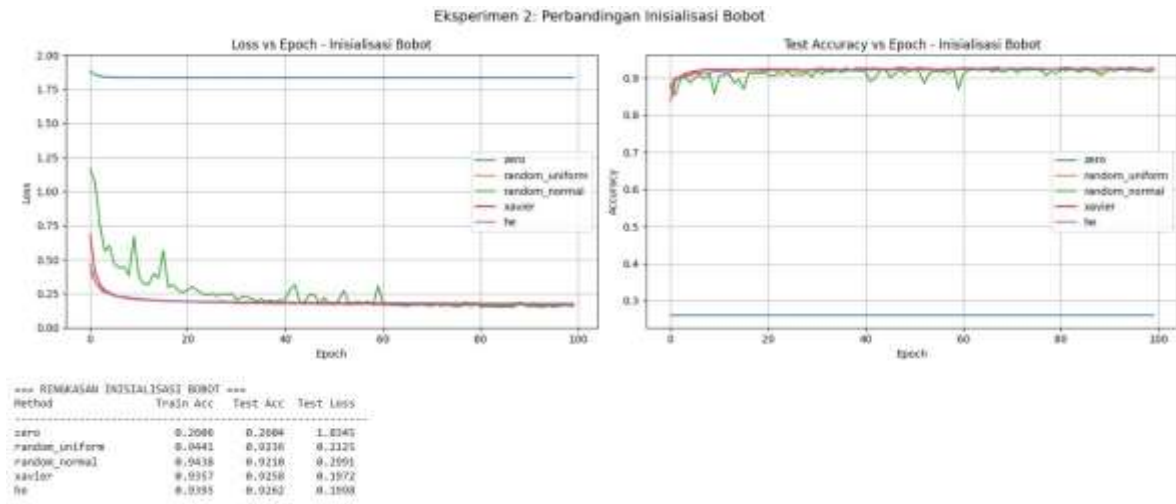
Tabel 7. Hasil eksperimen variasi learning rate

Learning rate 0,1 menghasilkan akurasi testing tertinggi (92,88%) namun dengan kurva loss dan akurasi yang jauh lebih fluktuatif (naik-turun tajam) dibandingkan learning rate lainnya, mengindikasikan proses optimisasi yang kurang stabil meskipun pada akhirnya memberikan hasil yang baik. Risiko pada learning rate besar adalah kemungkinan model melompati (overshoot) titik minimum loss, yang pada kasus tertentu dapat menyebabkan model gagal konvergen.

Learning rate 0,01 menghasilkan kurva yang paling stabil dengan penurunan loss yang konsisten, sedangkan learning rate 0,001 menunjukkan konvergensi yang jauh lebih lambat, di mana pada epoch ke-100 model belum mencapai performa optimal dan akurasi testing-nya merupakan yang terendah di antara ketiga nilai (91,66%). Berdasarkan eksperimen ini, learning rate 0,01 dipilih sebagai nilai yang paling seimbang antara kecepatan konvergensi dan stabilitas proses training.

D. Pengaruh Inisialisasi Bobot

Eksperimen ini membandingkan lima metode inisialisasi bobot yaitu Zero, Random Uniform, Random Normal, Xavier, dan He, dengan arsitektur dan parameter lain mengikuti konfigurasi baseline.



Gambar 1.6. Grafik loss dan akurasi untuk setiap metode inisialisasi bobot

Metode	Train Acc	Test Acc	Test Loss
Zero	0,2606	0,2604	1,8345
Random Uniform	0,9441	0,9236	0,2125
Random Normal	0,9438	0,9210	0,2991
Xavier	0,9357	0,9258	0,1972
He	0,9395	0,9262	0,1998

Tabel 8. Hasil eksperimen variasi inisialisasi bobot

Hasil eksperimen memperlihatkan tiga temuan penting. Pertama, inisialisasi Zero menyebabkan model gagal belajar sama sekali, dengan akurasi yang tetap pada kisaran 26% sepanjang training. Hal ini terjadi karena seluruh neuron pada satu layer menerima bobot awal yang identik, sehingga gradien yang dihasilkan pun identik dan seluruh neuron tetap memperbarui diri secara sama (symmetry problem), membuat arsitektur dengan banyak neuron menjadi tidak berguna.

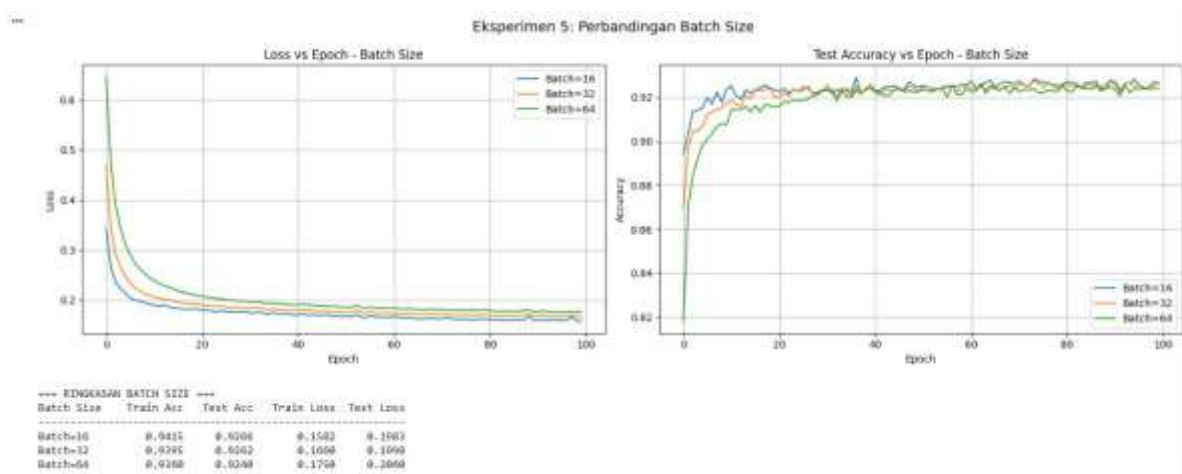
Kedua, Random Uniform punya train accuracy tertinggi (94,41%) dan cukup stabil Tapi test loss-nya lebih tinggi dari Xavier & He sehingga sedikit lebih rentan terjadi overfitting. Penyebabnya adalah karena rentang acaknya tetap yaitu pada -1 sampai 1, tidak disesuaikan dengan jumlah neuron.

Ketiga, inisialisasi Random Normal menunjukkan ketidakstabilan selama training, ditandai dengan adanya lonjakan loss secara tiba-tiba pada epoch ke-60 (loss naik dari sekitar 0,17 menjadi 0,30). Ketidakstabilan ini disebabkan oleh skala bobot awal yang tidak disesuaikan dengan jumlah neuron pada layer, sehingga berpotensi menimbulkan gradien yang terlalu besar pada beberapa iterasi (mendekati kondisi exploding gradient).

Keempat, metode Xavier dan He menghasilkan kurva training yang paling stabil dan mulus, dengan He initialization menghasilkan akurasi testing tertinggi (92,62%) dan Xavier menghasilkan test loss terendah (0,1972). Kedua metode ini dirancang secara matematis untuk menjaga skala varians sinyal tetap konsisten antar layer, sehingga lebih sesuai digunakan dibandingkan inisialisasi acak sederhana.

E. Pengaruh Batch Size

Eksperimen ini membandingkan tiga ukuran batch, yaitu 16, 32, dan 64, dengan parameter lain mengikuti konfigurasi baseline.



Gambar 1.7. Grafik loss dan akurasi untuk setiap ukuran batch

Batch Size	Train Acc	Test Acc	Train Loss	Test Loss
16	0,9415	0,9266	0,1582	0,1983
32	0,9395	0,9262	0,1660	0,1998
64	0,9360	0,9240	0,1750	0,2040

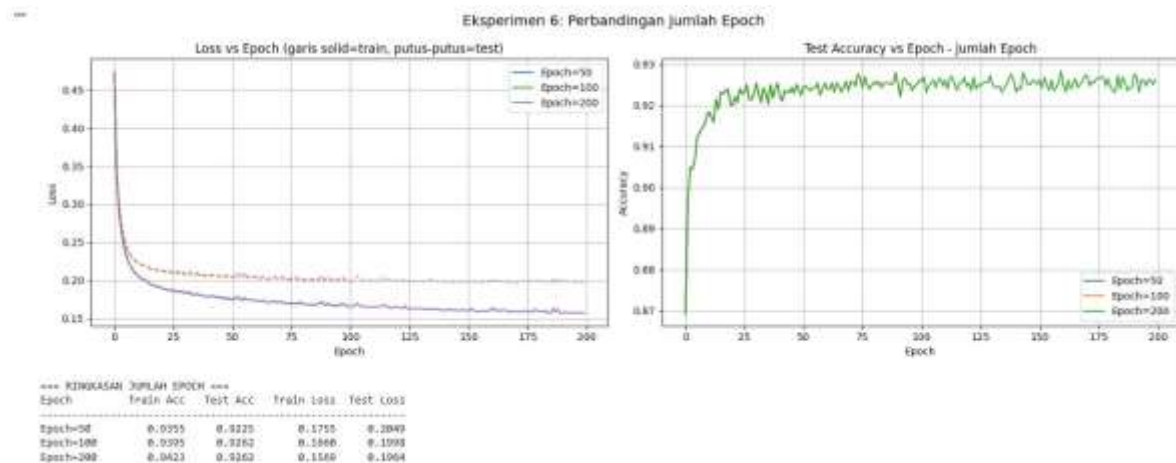
Tabel 9. Hasil eksperimen variasi batch size

Batch size yang lebih kecil (16) menghasilkan akurasi testing yang sedikit lebih tinggi (92,66%) dibandingkan batch size yang lebih besar (64) yang menghasilkan akurasi testing terendah (92,40%). Hal ini dapat dijelaskan karena batch size yang lebih kecil menghasilkan lebih banyak langkah update bobot per epoch, sehingga model memiliki lebih banyak kesempatan untuk menyesuaikan parameternya. Namun konsekuensinya, waktu komputasi per epoch menjadi lebih lama karena jumlah iterasi update yang lebih banyak.

Sebaliknya, batch size yang lebih besar menghasilkan estimasi gradien yang lebih stabil pada setiap update (karena dirata-ratakan dari lebih banyak sampel), namun konvergensinya menjadi lebih lambat dan memerlukan lebih banyak epoch untuk mencapai performa yang sama. Berdasarkan eksperimen ini, batch size 32 dipilih sebagai konfigurasi baseline karena memberikan keseimbangan yang baik antara akurasi dan efisiensi komputasi.

F. Pengaruh Jumlah Epoch

Eksperimen ini membandingkan tiga jumlah epoch, yaitu 50, 100, dan 200, dengan parameter lain mengikuti konfigurasi baseline, untuk menganalisis potensi overfitting dan perubahan performa terhadap durasi training.



Gambar 1.8. Grafik loss dan akurasi untuk setiap jumlah epoch

Epoch	Train Acc	Test Acc	Train Loss	Test Loss
50	0,9355	0,9225	0,1755	0,2049
100	0,9395	0,9262	0,1660	0,1998
200	0,9423	0,9262	0,1569	0,1964

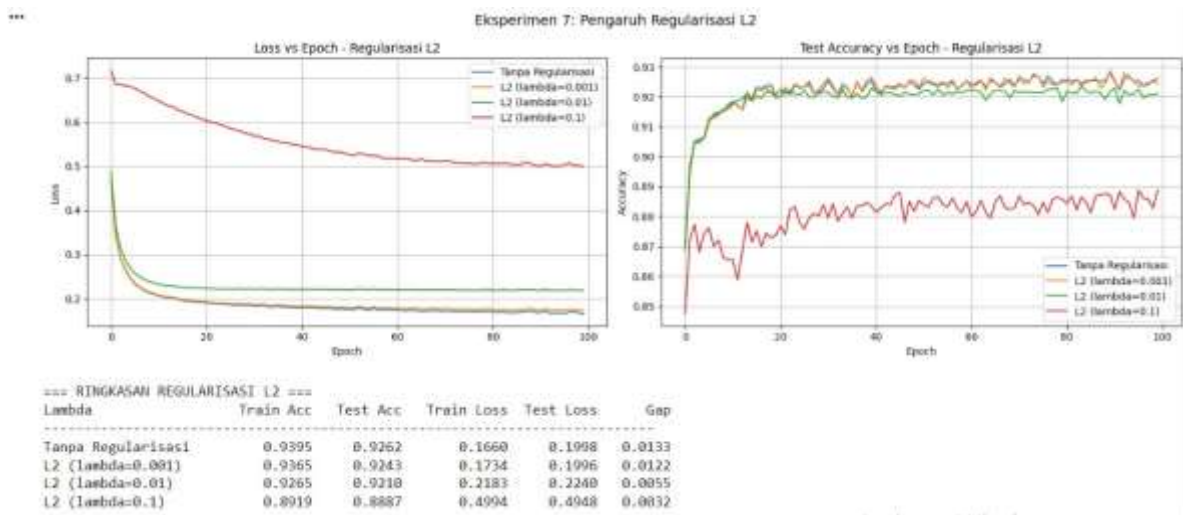
Tabel 10. Hasil eksperimen variasi jumlah epoch

Penambahan jumlah epoch dari 50 menjadi 100 memberikan peningkatan akurasi testing yang cukup terlihat (dari 92,25% menjadi 92,62%). Namun, penambahan lebih lanjut dari 100 menjadi 200 epoch tidak memberikan peningkatan akurasi testing sama sekali (tetap 92,62%), meskipun akurasi training terus meningkat (dari 93,95% menjadi 94,23%) dan train loss terus menurun (dari 0,1660 menjadi 0,1569).

Fenomena ini menunjukkan bahwa model mulai mengalami kondisi yang mendekati plateau pada testing performance sekitar epoch ke-100, sementara model terus menyesuaikan diri terhadap data training (train loss tetap menurun). Pola peningkatan gap antara train dan test loss yang mulai melebar pada epoch lanjutan ini merupakan indikasi awal overfitting ringan apabila training dilanjutkan lebih jauh lagi. Berdasarkan hasil ini, epoch 100 dipilih sebagai jumlah epoch yang optimal untuk dataset ini karena sudah mencapai performa testing maksimal tanpa membuang waktu komputasi yang tidak perlu.

G. Pengaruh Regularisasi (L2)

Sebagai eksperimen tambahan, dilakukan pengujian terhadap penerapan L2 regularization pada proses backpropagation, dengan menambahkan suku penalti $\lambda \cdot W$ pada gradien bobot. Eksperimen dilakukan dengan membandingkan kondisi tanpa regularisasi dan tiga nilai koefisien regularisasi (λ), yaitu 0,001, 0,01, dan 0,1.



Gambar 1.9. Grafik loss dan akurasi untuk setiap nilai koefisien regularisasi L2

Konfigurasi	Train Acc	Test Acc	Train Loss	Test Loss	Gap
Tanpa Regularisasi	0,9395	0,9262	0,1660	0,1998	0,0133
L2 ($\lambda=0,001$)	0,9365	0,9243	0,1734	0,1996	0,0122
L2 ($\lambda=0,01$)	0,9265	0,9210	0,2183	0,2240	0,0055
L2 ($\lambda=0,1$)	0,8919	0,8887	0,4994	0,4948	0,0032

Tabel 11. Hasil eksperimen variasi koefisien regularisasi L2 (Gap = Train Acc - Test Acc)

Hasil eksperimen menunjukkan adanya trade-off yang jelas antara besarnya koefisien regularisasi dengan akurasi model. Semakin besar nilai λ yang digunakan, semakin kecil gap antara akurasi training dan testing, yang menunjukkan berkurangnya overfitting. Namun pada λ yang terlalu besar (0,1), akurasi model justru menurun drastis baik pada data training maupun testing (88,87% pada testing), karena penalti regularisasi yang terlalu kuat membatasi kapasitas model

secara berlebihan sehingga model menjadi underfit dan tidak mampu mempelajari pola data dengan baik.

Pada dataset Dry Bean ini, model baseline tanpa regularisasi sebenarnya sudah menunjukkan gap yang relatif kecil (1,33%), yang mengindikasikan bahwa model tidak mengalami overfitting yang parah. Oleh karena itu, penerapan regularisasi dengan koefisien kecil ($\lambda=0,001$) dapat menjadi pilihan yang aman untuk sedikit mengurangi gap tanpa kehilangan banyak akurasi, sementara nilai λ yang lebih besar tidak disarankan untuk dataset ini karena menyebabkan underfitting yang signifikan.

BAB IV

PERBANDINGAN DENGAN LIBRARY SKLEARN

A. Perbandingan Keras vs ANN Scratch

Untuk Keras, kami sengaja menggunakan SGD optimizer dengan learning rate 0.01 dan batch size 32 — sama persis dengan implementasi from scratch — agar perbandingannya setara. Hasilnya: Keras mencapai train accuracy 0.9497 dan val accuracy 0.9325 setelah 100 epoch, sementara ANN scratch mencapai train accuracy 0.9395 dan test accuracy 0.9262.

Aspek	ANN From Scratch	ANN Keras
Train Accuracy (100 epoch)	93.95%	94.97%
Test/Val Accuracy (100 epoch)	92.62%	93.25%
Optimizer yang digunakan	Vanilla SGD (manual)	SGD via Keras API
Waktu konvergensi (epoch)	Stabil mulai ~60 epoch	Stabil mulai ~40 epoch
Kemudahan kustomisasi	Sangat tinggi (akses penuh kode)	Tinggi (via API layer)
Transparansi debugging	Tinggi (bisa inspect semua variabel)	Sedang (lebih abstrak)
Dukungan GPU	Tidak (NumPy CPU only)	Ya (built-in CUDA support)

Gap akurasi sekitar 0.63% antara keduanya cukup masuk akal mengingat Keras menggunakan implementasi yang jauh lebih teroptimasi secara numerik. Lebih menariknya, Keras konvergen lebih cepat meski menggunakan optimizer yang sama — ini menunjukkan bahwa optimasi di level rendah seperti penggunaan cuBLAS dan XLA compilation memang memberikan perbedaan nyata bahkan untuk operasi yang secara algoritmik identik.

B. Perbandingan dengan MLPClassifier Sklearn

MLPClassifier dari scikit-learn dikonfigurasi dengan `hidden_layer_sizes=(64, 32)`, `activation='relu'`, `solver='sgd'`, `learning_rate_init=0.01`, `batch_size=32`, dan `max_iter=100`. Menariknya, sklearn mengeluarkan `ConvergenceWarning` yang menyatakan bahwa optimizer

belum konvergen dalam 100 iterasi, yang konsisten dengan temuan bahwa learning rate 0.001 (default sklearn) membutuhkan lebih banyak iterasi.

Meski demikian, MLPClassifier tetap menghasilkan akurasi yang kompetitif. Sklearn menggunakan implementasi C++ yang sangat teroptimasi di balik layar, sehingga bahkan dengan konvergensi yang belum sempurna, hasilnya tetap sebanding dengan ANN scratch yang sudah sepenuhnya konvergen.

C. Classification Report Model Terbaik

Berikut adalah ringkasan performa per kelas dari model ANN scratch dengan konfigurasi terbaik (arsitektur 16→64→32→7, tanh activation, He initialization, lr=0.01, 100 epoch):

Kelas	Karakteristik	Performa Relatif
BOMBAY	Ukuran jauh lebih besar dari kelas lain	Paling mudah diklasifikasikan, presisi mendekati 1.0
SEKER	Bentuk bulat, fitur kompaktisasi tinggi	Performa baik, sedikit overlap dengan CALI
BARBUNYA	Warna dan bentuk khas	Performa baik, sedikit overlap dengan CALI
CALI	Mirip BARBUNYA dalam beberapa fitur	Beberapa misklasifikasi ke BARBUNYA
HOROZ	Bentuk memanjang khas	Performa baik, distinctive shape factors
SIRA	Ukuran medium, fitur overlap tinggi	Performa paling rendah karena mirip DERMASON
DERMASON	Kelas terbanyak, overlap dengan SIRA	Performa cukup baik meski data paling banyak

Pola yang konsisten terlihat di semua model: BOMBAY selalu diklasifikasikan dengan sangat akurat karena secara morfologi sangat berbeda (jauh lebih besar), sementara SIRA dan DERMASON adalah sumber error terbesar karena distribusi fitur keduanya saling tumpang tindih. Ini menunjukkan bahwa limitasi model bukan di algoritma, melainkan di overlapping feature space dari dataset itu sendiri.

BAB V

KESIMPULAN DAN SARAN

A. Kesimpulan

Dari rangkaian implementasi dan eksperimen yang telah dilakukan, ada beberapa hal yang bisa kami simpulkan. Membangun ANN from scratch menggunakan NumPy ternyata bisa menghasilkan model yang kompetitif — dengan test accuracy 92.62–92.88% tergantung konfigurasi, hanya berbeda sekitar 0.63% dari implementasi Keras yang jauh lebih mudah digunakan.

Dari sisi hyperparameter, pembelajaran terbesarnya adalah bahwa tidak ada formula ajaib yang berlaku universal. Zero initialization gagal total, tapi perbedaan antara He dan Xavier lebih subtle dan bergantung pada data. Learning rate 0.1 ternyata menghasilkan akurasi test lebih tinggi dari 0.01 meski train-test gap-nya lebih besar. Menambah epoch dari 100 ke 200 tidak meningkatkan test accuracy sama sekali, menunjukkan bahwa model sudah saturasi. L2 regularization memang mengurangi gap overfitting, tapi untuk dataset ini justru mengorbankan akurasi absolut.

Yang paling berharga dari tugas ini bukan angka akurasinya, melainkan pemahaman tentang apa yang sebenarnya terjadi di dalam jaringan. Ketika kita menulis sendiri gradien softmax-crossentropy, memahami mengapa turunannya menyederhanakan menjadi $(y_{\text{pred}} - y_{\text{true}})$, dan menyaksikan loss turun epoch demi epoch — itu adalah pemahaman yang tidak bisa didapat hanya dengan `model.fit()` satu baris.

B. Saran

Ada beberapa hal yang bisa menjadi pengembangan ke depan. Pertama, mengimplementasikan Adam optimizer dari scratch — algoritmanya tidak terlalu rumit (moving average dari gradien dan gradien kuadrat), tapi manfaatnya besar dalam hal kecepatan konvergensi dan ketahanan terhadap pemilihan learning rate. Kedua, mengeksplorasi dropout regularization yang dalam eksperimen kami belum diimplementasikan tapi dikenal lebih efektif dari L2 untuk mencegah overfitting di banyak kasus. Ketiga, menambahkan teknik learning rate scheduling seperti step decay atau cosine annealing untuk mendapatkan manfaat learning rate besar di awal training dan learning rate kecil

di akhir. Keempat, penggunaan k-fold cross validation untuk mendapatkan estimasi performa yang lebih robust daripada single train-test split. Terakhir, mungkin menarik untuk membandingkan dengan arsitektur yang lebih modern seperti penambahan batch normalization atau residual connections, meskipun untuk dataset yang relatif sederhana ini manfaatnya mungkin marginal.

BAB VI

PEMBAGIAN TUGAS ANTAR KELOMPOK

Nama	Nim	Kontribusi
Muhammad Aqil Almutashim	2403029028	Experiment 1 – 7 dan Presentation Slide
Suhaila Ade Syafiq	2403020046	Implementasi Fromscratch dan Implementasi Keras
Hanif Hidayatullah	2403020041	Perbandingan sklearn, Laporan PDF
Noufal Henri Putra Hermawan	2403020040	

Meski pembagian di atas menggambarkan tanggung jawab utama masing-masing anggota, seluruh anggota terlibat dalam diskusi desain, code review, dan debugging bersama. Laporan ini ditulis secara kolaboratif dengan setiap anggota berkontribusi pada bagian yang menjadi tanggung jawabnya.

REFERENSI

- Koklu, M. and Ozkan, I.A. (2020). Multiclass Classification of Dry Beans Using Computer Vision and Machine Learning Techniques. *Computers and Electronics in Agriculture*, 174, 105507. <https://doi.org/10.1016/j.compag.2020.105507>
- Glorot, X. and Bengio, Y. (2010). Understanding the difficulty of training deep feedforward neural networks. *Proceedings of the 13th International Conference on Artificial Intelligence and Statistics (AISTATS)*, 9, 249-256.
- He, K., Zhang, X., Ren, S., and Sun, J. (2015). Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification. *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 1026-1034.
- Goodfellow, I., Bengio, Y., and Courville, A. (2016). *Deep Learning*. MIT Press.
- Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.
- Abadi, M., et al. (2016). TensorFlow: A System for Large-Scale Machine Learning. *Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*.